Scenario

You are a security professional working at a health care company. As part of your job, you're required to regularly update a file that identifies the employees who can access restricted content. The contents of the file are based on who is working with personal patient records. Employees are restricted access based on their IP address. There is an allow list for IP addresses permitted to sign into the restricted subnetwork. There's also a remove list that identifies which employees you must remove from this allow list.

Your task is to create an algorithm that uses Python code to check whether the allow list contains any IP addresses identified on the remove list. If so, you should remove those IP addresses from the file containing the allow list.

Algorithm for file updates in Python

Project description

As part of a security professional working at a health care company I am responsible for maintaining a file that identifies who has access to restricted contents. This is controlled with an allowed list of IP addresses inside a file named `"allow_list.txt"`. A separate list contains IP addresses of people who should no longer have access to restricted contents. I created an algorithm to automate updating the allowed list of IP addresses by removing the IP addresses in the remove list

Open the file that contains the allow list

For the first part of the algorithm I needed to import the text file into the code and since the text file is in the same folder as my jupyter notebook I didn't need specify the past for this I first assigned the file name as a string in a variable named `import_file` variable.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Then I used the `with` statement to open the file.

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

`with` statement is used with the `.open()` function in read mode to open the allow list file for the purpose of reading it and this will allow me to manage resources by closing the file after exciting the `with` statement. I then followed the code with the `open()` function, this function has two parameters. The first identifies the file to import to which I assigned the variable `import_file`, and then the second indicates what I want to do with the file. In this case, `"r"` indicates that I want to read it. The code also assigns the output of the code using the `as` keyword to a file I names this variable `file`; `file` stores the output of the `.open()` function.

Read the file contents

In order to read the contents of the file I used the `.read()` function.

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

Since my `.open()` dunction already incorporate the argument `"r"` for “read” I can call the `.read()` function directly in the body of the algorithm. This function converts the file into a string and allows my algorithm to read it. The `.read()` is implemented on the `file` valriable where I store the output earlier. I then assigned the converted string into the variable `ip_addresses`. This would allow me to later use the string output to organize and extract data from the text file.

Convert the string into a list

Since we only converted the contents of the file into a string output we still need to convert it into a list format. This enable us to change/modify the contents of the file. We need to do this since strings in python are immutable. Here I just called the `.split()` after the `ip_addresses` variable. I then stored the newly converted list back into the `ip_addresses` variable.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

Iterate through the remove list

Here I added a `for` loop to the algorithm, this will check the list if there are still IP addresses that should be removed since a for loop iterates trough a list as long as the set condition is met.

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:
```

Remove IP addresses that are on the remove list

Here inside my for loop I added a conditional if statement that checks if an element is in the remove_list. Then if the conditional statement is true the element needs to be removes from the IP addresses.

```
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        remove_list.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

Update the file with the revised list of IP addresses

To do the udate I used a `.join()` method this combines all items in an iterable into an string. After that I used another `with` statement with the `open()` function and a `.write()` method in order to update the file. The `.write()` method indicates that I want to open the file and write into the file

```
import_file = "allow_list.txt"
Lab workspace: Activity: Create another algorithm
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

Summary

In this scenario I created an algorithm that removes IP addresses from an allowed list. This is done by converting the file into a string and since strings in python are immutable the algorith then converts the file into a list. Then the algorithm uses a for loop that iterates through the items in the list. With each iteration an conditional statement runs inside the for loop this is to check if the IP address is in the remove list and if the IP address is in the list it is then removed using the `.remove()` method. After this I used the `.join()` method to convert the IP addresses back into a string so that I could write over the contents of the text file with the revised list of IP addresses.